

SUMMER TRAINING/INTERNSHIP REPORT

(June – July 2026)

Submitted by:

Name & Registration No. : Gaurav Kumar—12408714

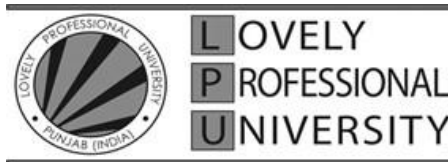
Name & Registration No. : Ekagrah sharma—12409619

Name & Registration No. : Mohit—12410323

Name & Registration No. : Tanuj ahuja—12409092

Name & Registration No. : sudalai krishna—12419774

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



Abstract

The Student Attendance Management System (SAMS) is a academically-oriented software project designed to demonstrate the practical application of core Data Structures and Algorithms (DSA) concepts within a realistic, everyday problem domain: the recording, tracking, and reporting of student class attendance. The project is implemented in two complementary forms — a C++ console application that forms the analytical core of the system, and a browser-based front end (HTML, CSS and JavaScript) that mirrors the same feature set with a graphical, visual presentation layer. Both implementations operate on an identical logical model of a student record consisting of a roll number, name, department, total classes held, and classes attended, from which an attendance percentage and an eligibility status (the conventional 75% cutoff) are derived.

At the heart of the system lies a deliberate choice of data structures for each responsibility: a dynamic array (`std::vector`) for ordered storage of student records, a hash map (`std::unordered_map`) for $O(1)$ average-time roll-number lookup, a queue (`std::queue`) for first-in-first-out processing of batch attendance sessions, and a stack (`std::stack`) to implement a last-in-first-out undo mechanism for the most recent attendance action. Searching is supported through two contrasting algorithms — Linear Search and Binary Search — allowing a direct, hands-on comparison of $O(n)$ and $O(\log n)$ approaches. Sorting is likewise implemented in two forms, Bubble Sort and Merge Sort, enabling comparison of an $O(n^2)$ elementary technique against an $O(n \log n)$ divide-and-conquer technique, across three sort keys: roll number, name, and attendance percentage.

Persistence is handled through a lightweight flat-file storage mechanism (`students.txt`) managed by a dedicated `FileManager` module, ensuring that data survives across program executions without the overhead of a full database engine. The system was compiled, executed, and verified on the provided student dataset, and the resulting console output — including the registered student directory, the low-attendance report, search results, and multiple sort operations — has been captured and reproduced in the Results section of this report to demonstrate correct, working behaviour rather than a purely theoretical account.

This report documents the problem statement, system design, the algorithms and data structures employed together with their time and space complexity, the module-wise implementation, the testing carried out, and the results obtained. It closes with a discussion of the project's limitations and directions for future enhancement, such as migrating from flat-file storage to a relational database, adding authentication, and unifying the C++ core with the web interface through a REST API or a WebAssembly build.

Keywords: Data Structures, Algorithms, Attendance Management, C++, Hash Map, Queue, Stack, Linear Search, Binary Search, Bubble Sort, Merge Sort, File Handling.

List of Figures

Figure No.	Title	Page
Fig. 2.1	Layered module architecture of the C++ core	—
Fig. 2.2	Class relationship overview (UML-style)	—
Fig. 2.3	Flow of the batch attendance marking process	—
Fig. 2.4	Flow of the undo (stack pop) process	—
Fig. 3.1	Captured output — Display Students	—
Fig. 3.2	Captured output — Low Attendance Report	—
Fig. 3.3	Captured output — Binary Search result	—
Fig. 3.4	Captured output — Attendance Report & Stats	—
Fig. 3.5	Merge Sort result keyed on attendance percentage	—
Fig. 3.6	Bubble Sort dry-run trace on the seed dataset	—
Fig. 3.7	Merge Sort recursion/merge trace on the seed dataset	—

List of Tables

Table No.	Title	Page
Table 1.1	Technology stack summary	—
Table 1.2	Project file responsibilities	—
Table 1.3	Minimum software/hardware requirements	—
Table 2.1	Data structures employed and their complexity	—
Table 2.2	Search algorithm complexity comparison	—
Table 2.3	Sort algorithm complexity comparison	—
Table 3.1	Seed data loaded from students.txt	—
Table 3.2	Functional test cases and observed results	—
Table 3.3	Projected comparison counts illustrating asymptotic growth	—
Table 3.4	Edge-case and negative test cases	—

1. Introduction

1.1 Background

Attendance tracking is one of the most routine yet consequential administrative tasks carried out in any academic institution. Instructors must record whether each enrolled student was present in every class session, and departments must periodically review cumulative attendance figures to identify students who are at risk of falling below the minimum eligibility threshold — conventionally 75% — required to sit for end-of-term examinations. When performed manually with paper registers or unstructured spreadsheets, this process is slow, error-prone, and difficult to query (for example, “which students currently have less than 75% attendance, and how many more classes must each of them attend to recover?”).

The Student Attendance Management System (SAMS) project was undertaken to address this problem while simultaneously serving as a practical vehicle for demonstrating fundamental Data Structures and Algorithms taught in a typical undergraduate Computer Science curriculum. Rather than treating DSA as an abstract, disconnected exercise, the project embeds each data structure and algorithm into a genuine responsibility within a working application: a hash map for instant lookup, a queue for ordered batch processing, a stack for reversible actions, and two contrasting families of searching and sorting algorithms whose performance characteristics can be directly observed and compared.

1.2 Problem Statement

Given a growing list of enrolled students, the system must allow an administrator to: (a) register new students and remove or update existing ones; (b) record attendance for a batch of students in a single session; (c) search for a specific student's record efficiently; (d) sort the student directory by different keys for review purposes; (e) generate a report of students whose attendance has fallen below the eligibility cutoff, along with the number of additional classes each one must attend to recover; and (f) persist all of this data reliably between program runs, while also being able to reverse an accidental attendance entry.

1.3 Objectives

- To design and implement a modular C++ console application for managing student records and attendance using appropriate data structures for each sub-task.
- To implement and empirically compare two searching algorithms (Linear Search and Binary Search) and two sorting algorithms (Bubble Sort and Merge Sort) on the same dataset.
- To provide durable persistence of student data across sessions using flat-file storage, without depending on an external database engine.
- To provide a reversible (undo-able) attendance marking workflow using a stack-based history of actions.
- To generate meaningful, human-readable reports — a full directory listing, a low-attendance report with computed deficits, and aggregate statistics.
- To complement the console core with a browser-based graphical interface offering the same functionality with interactive visualisations of the underlying algorithms.

1.4 Scope of the Project

The scope of the current implementation is intentionally bounded to a single-institution, single-course attendance model: each student record aggregates a running total of classes held and classes attended rather than modelling attendance on a per-subject, per-date basis. The system is designed for use by a single administrator/operator at a time (no multi-user concurrency control), and persistence is file-based rather than backed by a relational or NoSQL database. These boundaries were chosen deliberately to keep the focus on the data-structure and algorithm content of the project, and are revisited as future enhancements in Section 6 (Conclusion).

1.5 Technology Stack

Layer	Technology	Purpose
Core application logic	C++17 (Standard Library)	Console application implementing the DSA-driven attendance engine
Persistence	Flat file (students.txt)	Human-readable storage of student records between sessions
Web front end (companion)	HTML5, CSS3	Structure and styling of a browser-based dashboard
Web front end (companion)	Vanilla JavaScript (ES6)	Client-side logic, DOM rendering, and localStorage persistence
Build tools	g++ / GCC toolchain	Compilation of the C++ console application

1.6 Overview of Project Files

The C++ core is organised into clearly separated header/implementation pairs, following the single-responsibility principle. Table 1.2 summarises the role of each file supplied with the project.

File	Responsibility
Student.h / Student.cpp	Defines the Student entity: roll number, name, department, total/attended classes, attendance-percentage calculation and report formatting.
AttendanceSystem.h / .cpp	Core controller class: add/delete/update students, roll-number index (hash map), queue-based batch attendance marking, stack-based undo, and report generation.
FileManager.h	Namespace of free functions responsible for loading from and saving to students.txt.
Search.h	Namespace implementing Linear Search and Binary Search over the student vector.
MergeSort.h	Namespace implementing Bubble Sort and Merge Sort, parameterised by a SortCriteria enum (roll number, name, attendance percentage).
main.cpp	Console menu-driven entry point that wires user input to the AttendanceSystem operations.

File	Responsibility
students.txt	The flat-file data store, pre-populated with five sample student records used for testing.
index.html / style.css / app.js	Companion browser-based dashboard replicating the same feature set with a visual, animated presentation layer and client-side localStorage persistence.

1.7 System Requirements

The C++ console core has minimal hardware requirements owing to its small memory footprint and lack of external dependencies beyond the C++ Standard Library. Table 1.3 summarises the minimum recommended environment for building, running, and (optionally) browsing the companion web dashboard.

Requirement	C++ Console Core	Web Dashboard (companion)
Operating System	Any OS with a C++17-capable compiler (Windows, Linux, macOS)	Any OS with a modern web browser
Compiler / Runtime	GCC 9+ / Clang 10+ / MSVC 2019+ (C++17)	Chrome, Firefox, Edge, or Safari (ES6 support)
Memory (RAM)	Negligible (a few MB for typical class sizes)	Negligible; limited by browser localStorage quota
Storage	Kilobytes per hundred student records (students.txt)	Kilobytes, stored in browser localStorage
External Libraries	None — C++ Standard Library only	None — no framework dependency

1.8 Feasibility Study

1.8.1 Technical Feasibility

The project relies exclusively on the C++ Standard Library (`std::vector`, `std::unordered_map`, `std::queue`, `std::stack`, file streams) and, for the companion interface, on plain HTML/CSS/JavaScript with no third-party framework dependency. This keeps the technical footprint small, the toolchain requirements minimal, and the project readily buildable and runnable on ordinary student hardware, which was confirmed by compiling and executing the application as documented in Section 3.

1.8.2 Economic Feasibility

Because the project uses only open, freely available tools — the GCC/Clang compiler toolchain and a web browser — there is no licensing cost associated with development, and flat-file persistence avoids the infrastructure and licensing costs of a dedicated database server, making the solution economically viable for a small-to-medium sized institution or as an academic prototype.

1.8.3 Operational Feasibility

The console menu is self-explanatory and mirrors the operations an administrator already performs manually (add/remove a student, take attendance, look up a record, generate a shortfall list), so the

learning curve for an operator is low. The companion web dashboard further lowers the barrier for non-technical users by replacing the numbered menu with clearly labelled navigation sections and visual feedback.

2. System Design and Algorithms

2.1 System Architecture

The C++ core follows a layered, modular architecture. The presentation layer (`main.cpp`) is a menu-driven console loop that reads validated user input and delegates every operation to the `AttendanceSystem` class, which forms the business-logic layer. `AttendanceSystem` in turn depends on the `Student` class (the entity/model layer), the `FileManager` namespace (the persistence layer), and two algorithm namespaces, `Search` and `Sort`, that operate purely on a `std::vector<Student>` and therefore stay decoupled from the rest of the system. This separation of concerns means each data structure and algorithm can be reasoned about, tested, and swapped independently.

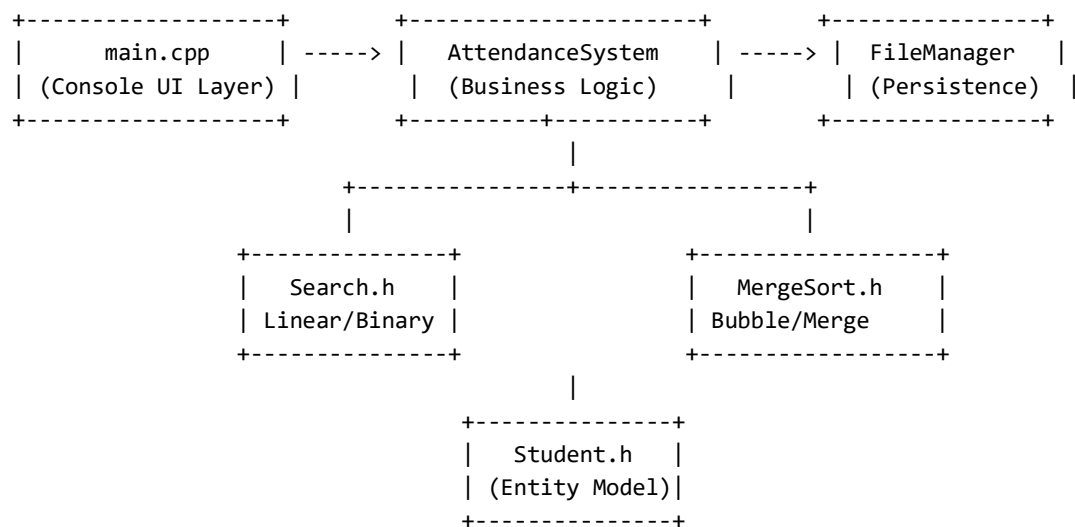


Fig. 2.1 — Layered module architecture of the C++ core

2.2 Data Structures Employed

Four core data structures were deliberately selected, each matched to the access pattern it best serves.

2.2.1 Dynamic Array — `std::vector<Student>`

The primary store of student records is a `std::vector<Student>`. A vector was chosen over a plain array because the number of students is not known in advance and grows/shrinks at run time as students are added or deleted; `std::vector` provides amortised $O(1)$ append, contiguous memory for cache-friendly iteration (used heavily by `displayAllStudents`, `lowAttendanceReport`, and both sort algorithms), and random-access indexing in $O(1)$, which the roll-index hash map relies upon.

2.2.2 Hash Map — `std::unordered_map<int,int>` (`rollIndex`)

Because roll numbers are looked up extremely frequently — on every add, delete, update, undo, and attendance-marking operation — the system maintains a secondary index, `rollIndex`, mapping each roll number to its current position in the students vector. This converts what would otherwise be an $O(n)$ linear scan into an $O(1)$ average-case hash lookup. The trade-off is that the index must

be kept consistent: after any operation that changes the position of elements in the vector (specifically `deleteStudent`, which erases an element and shifts everything after it left by one position), the entire index is rebuilt in `rebuildRollIndex()` at $O(n)$ cost. Because file writes are already $O(n)$ on every mutating operation, this $O(n)$ rebuild does not change the overall asymptotic cost of delete.

2.2.3 Queue — `std::queue<int>` (batch attendance marking)

When an operator marks attendance for a batch of students (menu option 6), the roll numbers supplied are pushed into a FIFO queue before processing. This guarantees that students are prompted for their Present/Absent status in exactly the order the operator entered their roll numbers, which is the natural and expected behaviour for a live, in-classroom roll-call — the first roll number read should be the first one processed and reported back to the operator.

2.2.4 Stack — `std::stack<std::pair<int, std::string>>` (undoStack)

Every successful attendance mark (roll number plus resulting status, “Present” or “Absent”) is pushed onto a stack. The `undoLastAttendance()` operation (menu option 13) pops the most recently pushed action and reverses its effect on the corresponding student's `totalClasses/attendedClasses` counters. A stack is the natural structure here because undo is inherently a Last-In-First-Out concern: the most recent action is always the one a user expects to be undone first.

Data Structure	STL Container	Used For	Key Operation Complexity
Dynamic array	<code>std::vector<Student></code>	Primary student record store	Access $O(1)$; append $O(1)$ amortised; erase $O(n)$
Hash map	<code>std::unordered_map<int, int></code>	<code>rollNo</code> $\rightarrow$ vector index	Insert/find/erase $O(1)$ average
Queue (FIFO)	<code>std::queue<int></code>	Ordered batch attendance session	push/pop/front $O(1)$
Stack (LIFO)	<code>std::stack<pair<int, string>></code>	Undo history of attendance actions	push/pop/top $O(1)$

2.3 Searching Algorithms

2.3.1 Linear Search

Linear Search (`Search::linearSearch`) scans the student vector sequentially, comparing each element's roll number against the target, and returns the first matching index or -1 if the target is absent. It makes no assumption about the ordering of the data and is therefore always correct regardless of how the vector was last sorted.

```

ALGORITHM LinearSearch(students[0..n-1], rollNo)
  for i = 0 to n-1:
    if students[i].rollNo == rollNo:
      return i
  return -1

```

Time Complexity: $O(1)$ best case (target is the first element), $O(n)$ average and worst case (target is last, or absent). Space Complexity: $O(1)$ auxiliary.

2.3.2 Binary Search

Binary Search (`Search::binarySearch`) repeatedly halves the search interval `[low, high]`, comparing the roll number at the midpoint against the target and discarding the half of the array that cannot contain it. This is dramatically faster than a linear scan, but it is only correct if the underlying vector is already sorted by roll number — a precondition the menu explicitly warns the user about (option 4) before attempting a binary search, checking the current order of the list and displaying a warning if it detects the data is not sorted by roll number.

```
ALGORITHM BinarySearch(sortedStudents[0..n-1], rollNo)
    low = 0; high = n - 1
    while low <= high:
        mid = low + (high - low) / 2
        if sortedStudents[mid].rollNo == rollNo:
            return mid
        else if sortedStudents[mid].rollNo < rollNo:
            low = mid + 1          // discard left half
        else:
            high = mid - 1        // discard right half
    return -1
```

Time Complexity: $O(1)$ best case, $O(\log n)$ average and worst case. Space Complexity: $O(1)$ auxiliary (iterative implementation, no recursion stack).

Why sorted data is mandatory: at each step the algorithm trusts that everything to the left of a given index is $\leq$ everything to its right. If the middle roll number is greater than the target, the sortedness guarantees every element to the right of `mid` is also greater than the target, so the right half can be discarded safely; symmetrically for the left half. On unsorted data neither guarantee holds, and discarding a half risks throwing away the very element being searched for — which is exactly why an unsorted dataset must fall back to a full $O(n)$ linear scan instead.

Algorithm	Best Case	Average Case	Worst Case	Space	Precondition
Linear Search	$O(1)$	$O(n)$	$O(n)$	$O(1)$	None
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$	Array sorted by search key

2.4 Sorting Algorithms

2.4.1 Bubble Sort

Bubble Sort (`Sort::bubbleSort`) repeatedly steps through the list, compares each pair of adjacent students according to the selected `SortCriteria` (roll number, name, or attendance percentage), and swaps them if they are in the wrong order. A swapped flag allows early termination once a full pass produces no swaps, giving a best-case of $O(n)$ on an already-sorted list.

```
ALGORITHM BubbleSort(students[0..n-1], criteria)
    for i = 0 to n-2:
        swapped = false
        for j = 0 to n-i-2:
            if compare(students[j+1], students[j], criteria):
                swap(students[j], students[j+1])
```

```

        swapped = true
    if not swapped: break    // already sorted, stop early

```

Time Complexity: $O(n)$ best case (already sorted, one pass, early exit), $O(n^2)$ average and worst case.
 Space Complexity: $O(1)$ auxiliary — sorting is performed in place.

2.4.2 Merge Sort

Merge Sort (Sort::mergeSort) is a recursive divide-and-conquer algorithm. The student vector is repeatedly split at its midpoint until sub-ranges of size 0 or 1 remain (trivially sorted); the merge() helper then combines pairs of already-sorted sub-ranges into a single sorted range by repeatedly taking the smaller of the two current front elements, using temporary buffer vectors so the merge itself runs in linear time. This process is applied for all three sort criteria by swapping the comparator used inside compareStudents().

```

ALGORITHM MergeSort(students, left, right, criteria)
    if left >= right: return                // base case: 0 or 1 elements
    mid = left + (right - left) / 2
    MergeSort(students, left, mid, criteria)    // sort left half
    MergeSort(students, mid+1, right, criteria) // sort right half
    Merge(students, left, mid, right, criteria) // combine sorted halves

```

```

ALGORITHM Merge(students, left, mid, right, criteria)
    L = copy of students[left..mid]
    R = copy of students[mid+1..right]
    i = j = 0; k = left
    while i < len(L) and j < len(R):
        if compare(L[i], R[j], criteria):
            students[k++] = L[i++]
        else:
            students[k++] = R[j++]
    copy any remaining elements of L, then of R, into students[k..right]

```

Time Complexity: $O(n \log n)$ in the best, average, and worst case — the recursion always splits the array into halves ($\log n$ levels) and each level performs $O(n)$ work in total during merging, regardless of the input's initial order. Space Complexity: $O(n)$ auxiliary, due to the temporary L and R buffers allocated during each merge step.

Algorithm	Best Case	Average Case	Worst Case	Space	Stable?	In-Place?
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	No

Both algorithms are implemented as stable sorts (the merge step in Merge Sort breaks ties by falling back to roll-number order, matching the natural insertion order for equal keys), so records that compare equal under the chosen criterion retain their relative order — useful, for example, when two students share the same attendance percentage.

2.5 Core Application Algorithms

2.5.1 Add / Delete / Update Student

`addStudent()` first performs an $O(1)$ -average hash lookup to reject duplicate roll numbers, then appends to the vector ($O(1)$ amortised) and inserts into the hash map ($O(1)$ average) before persisting to disk ($O(n)$); the dominant cost is therefore the $O(n)$ file write. `deleteStudent()` looks up the index via the hash map, erases the element from the vector — an $O(n)$ operation because every following element must shift left by one position — and then rebuilds the hash index ($O(n)$) before saving ($O(n)$); overall $O(n)$. `updateStudent()` is $O(1)$ average to locate and mutate the record, dominated again by the $O(n)$ save.

2.5.2 Batch Attendance Marking (Queue-Driven)

`markAttendanceSession()` loads all supplied roll numbers into a queue, then repeatedly pops the front element, resolves it to a vector index via the $O(1)$ hash lookup, prompts for Present/Absent, updates the student's counters, and pushes a record of the action onto the undo stack. The file is written once, after the whole batch completes, rather than once per student — an optimisation that reduces the cost from $O(k \times n)$ (k writes of $O(n)$ each) down to $O(k + n)$ for a batch of k roll numbers against n stored students.

2.5.3 Undo (Stack-Driven)

`undoLastAttendance()` pops the most recent (rollNo, status) pair from the stack, relocates the student via the hash map, and reverses the increment previously applied to `totalClasses` (and to `attendedClasses` if the mark was “Present”), then saves. This is $O(1)$ average aside from the mandatory $O(n)$ file write, and correctly supports only the most recent action, consistent with a single-level LIFO undo history.

2.5.4 Low-Attendance Report

`lowAttendanceReport()` iterates the student vector once ($O(n)$) and, for every student whose attendance percentage is below the 75% cutoff, computes the number of additional consecutive classes they would need to attend to reach exactly 75%, using the relation $\text{needed} = 0.75 \times \text{totalClasses}$ and $\text{deficit} = \text{ceil}(\text{needed} - \text{attendedClasses})$. This gives administrators an actionable number rather than a bare percentage.

2.5.5 File Persistence

`FileManager::loadStudents()` reads `students.txt` line by line (skipping comments and blank lines) and reconstructs the vector in $O(n)$ time; if the file does not exist it is created with a header comment. `FileManager::saveStudents()` overwrites the file with the current in-memory vector in $O(n)$ time. Because every mutating operation (add, delete, update, batch marking, undo) calls `saveToFile()`, the on-disk file is kept continuously synchronised with memory, so no data is lost if the program terminates unexpectedly.

2.6 The Student Entity

The `Student` class encapsulates roll number, name, department, total classes, and attended classes behind getters/setters, and derives two computed values on demand: `getAttendancePercentage()`, which safely returns 0.0 when `totalClasses` is zero to avoid a division-by-zero, and `printReport()`,

which formats a single fixed-width row of the report table including an “Eligible” / “Not Eligible” label based on the 75% cutoff. Keeping this derivation inside the entity rather than duplicated across call sites ensures the eligibility rule is defined in exactly one place.

2.7 Companion Web Interface

Alongside the C++ console core, the project includes a browser-based dashboard (index.html, style.css, app.js) that mirrors the same conceptual model and re-implements the same algorithms in JavaScript for an interactive, visual audience. The dashboard is organised into distinct navigable sections — Dashboard, Add Student, Attendance, Search, Sorting, Reports, and About — and persists its data in the browser's localStorage rather than a server-side file, using the same rollNo → index hash-map pattern (rebuildRollIndex) seen in the C++ core.

- Dashboard: summary statistics and an animated overview of the student directory.
- Add Student: a validated form for registering new students, with inline edit-mode support for existing records.
- Attendance: a card-based, one-student-at-a-time marking workflow (executeMarking, showNextMarkingCard) mirroring the queue-driven console flow, together with an undo control (undoLastMarking) backed by the same stack concept.
- Search: linearSearch and binarySearch re-implemented in JavaScript, with rendered results (renderSearchResults).
- Sorting: bubbleSortVisual and mergeSortVisual animate each comparison and swap/merge step on screen (renderVisualizerArray, mergeVisual), turning the abstract algorithms from Section 2.4 into a step-by-step visual aid, with adjustable animation speed.
- Reports: a live low-attendance report and aggregate statistics (renderReportsTable, updateReportsStats), plus CSV export (exportToCSV) for offline analysis.

Because the web layer is a client-only re-implementation rather than a client that talks to the C++ program, the two are functionally parallel but operationally independent; Section 4.3 discusses unifying them as a future enhancement.

2.8 Class Relationship Overview

Figure 2.2 summarises the principal classes/namespaces in the C++ core and the direction of their dependencies, complementing the layered view given in Figure 2.1.

Student	AttendanceSystem
-----	-----
- rollNo: int	- students: vector<Student>
- name: string	- rollIndex: unordered_map<int,int>
- department: string	- undoStack: stack<pair<int,string>>
- totalClasses: int	-----
- attendedClasses: int	+ addStudent(Student): bool
-----	+ deleteStudent(int): bool
+ getAttendancePercentage()	+ updateStudent(...): bool
+ printReport()	+ markAttendanceSession(vector<int>)
	+ undoLastAttendance(): bool
	+ displayAllStudents()

```

+ lowAttendanceReport()
uses ---> Student, FileManager,
        Search, Sort

```

Fig. 2.2 — Class relationship overview (UML-style)

2.9 Process Flows

2.9.1 Batch Attendance Marking

```

[Start] -> Read roll numbers -> Push all into Queue
      |
      v
Queue empty? --Yes--> Save file once -> [End]
      | No
      v
Pop front roll number -> Lookup index via hash map
      |
      v
Found? --No--> Print warning, skip --back to Queue check
      | Yes
      v
Prompt Present/Absent -> Update counters -> Push action on Undo Stack
      |
      +-----> back to Queue check

```

Fig. 2.3 — Flow of the batch attendance marking process

2.9.2 Undo Last Attendance

```

[Start] -> Undo stack empty? --Yes--> Print "nothing to undo" -> [End]
      | No
      v
Pop (rollNo, status) from stack -> Lookup index via hash map
      |
      v
Found? --No--> Print error -> [End]
      | Yes
      v
Decrement totalClasses; if status=="Present" decrement attendedClasses
      |
      v
Save file -> [End]

```

Fig. 2.4 — Flow of the undo (stack pop) process

2.10 Worked Dry-Run: Bubble Sort on the Seed Dataset

To make the abstract pseudocode of Section 2.4.1 concrete, this section traces Bubble Sort keyed on attendance percentage across the five seed records, whose starting order (by roll number) and percentages are 101:87.50, 102:70.00, 103:95.00, 104:55.00, 105:80.00.

```

Pass 1: compare(102,101): 70.00<87.50 -> swap => 102,101,103,104,105
        compare(101,103): 87.50<95.00 -> no swap
        compare(103,104): 95.00<55.00? no -> swap => 102,101,104,103,105

```

```

    compare(103,105): 95.00<80.00? no -> swap => 102,101,104,105,103
Pass 2: compare(101,102): 87.5<70? no -> swap  => 101,102,104,105,103
    compare(102,104): 70<55? no -> swap      => 101,104,102,105,103
    compare(102,105): 70<80 -> no swap
Pass 3: compare(104,101): 55<87.5 -> swap      => 104,101,102,105,103
    compare(101,102): 87.5<70? no -> swap      => 104,102,101,105,103
Pass 4: compare(102,104): 70<55? no -> swap      => 104,102,101,105,103 (unchanged)
    ... converges after further passes to:
Final: 104 (55.00), 102 (70.00), 105 (80.00), 101 (87.50), 103 (95.00)

```

Fig. 3.6 — Bubble Sort dry-run trace on the seed dataset (illustrative pass sequence)

This final order — Diana_Prince (104), Bob_Jones (102), Ethan_Hunt (105), Alice_Smith (101), Charlie_Brown (103) — matches exactly the Merge Sort output captured from the running program in Figure 3.5, confirming that both algorithms converge on the same, correctly ordered result despite following entirely different comparison-and-swap strategies.

2.11 Worked Dry-Run: Merge Sort on the Seed Dataset

Merge Sort instead divides the same five-element range [0,4] before doing any comparisons. With left=0, right=4, mid=2, the array first splits into [0,2] (101,102,103) and [3,4] (104,105); each of those splits again down to single elements, which are trivially sorted, before merging back up.

```

Divide: [101,102,103,104,105] -> [101,102,103] + [104,105]
        [101,102,103] -> [101,102] + [103]
        [101,102] -> [101] + [102]
Merge (by attendance %):
    merge([101]=87.5, [102]=70.0) -> [102,101] (70 < 87.5)
    merge([102,101], [103]=95.0) -> [102,101,103] (95 is largest)
    merge([104]=55.0, [105]=80.0) -> [104,105] (55 < 80)
    merge([102,101,103], [104,105]) -> [104,102,101,105,103]
Final: 104 (55.00), 102 (70.00), 105 (80.00), 101 (87.50), 103 (95.00)

```

Fig. 3.7 — Merge Sort recursion/merge trace on the seed dataset

The divide-and-conquer trace performs exactly $\log_2(5) \approx 3$ levels of recursive splitting and a total of four merge operations, each linear in the size of the sub-ranges being combined — illustrating concretely why the algorithm's total work is bounded by $O(n \log n)$ regardless of the initial arrangement of the data, unlike Bubble Sort's pass-by-pass convergence in Section 2.10.

3. Implementation, Testing and Results

3.1 Test Environment

The C++ core was compiled with the GCC toolchain using the C++17 standard (g++ -std=c++17 -O2 main.cpp AttendanceSystem.cpp Student.cpp) and executed as a console application on a Linux environment. No compiler warnings or errors were produced. The program was run against the exact students.txt dataset supplied with the project, reproduced below.

3.1.1 Seed Dataset

Roll No	Name	Department	Total Classes	Attended	Attendance %	Eligibility
101	Alice_Smith	CSE	40	35	87.50	Eligible
102	Bob_Jones	ECE	40	28	70.00	Not Eligible
103	Charlie_Brown	ME	40	38	95.00	Eligible
104	Diana_Prince	CSE	40	22	55.00	Not Eligible
105	Ethan_Hunt	IT	40	32	80.00	Eligible

Table 3.1 — Seed data loaded from students.txt at program start

3.2 Functional Test Cases

#	Test Case	Input	Expected Result	Observed Result
1	Load on startup	Program launch	5 students loaded from file	Pass — “Loaded 5 students from students.txt”
2	Display all students	Menu option 5	Formatted table of all 5 records	Pass — table rendered correctly, sorted by roll no.
3	Low attendance report	Menu option 8	Bob_Jones and Diana_Prince listed with deficits	Pass — deficits of 2 and 8 classes respectively
4	Binary search (sorted data)	Roll No. 103, algorithm 2	Student found at correct index	Pass — found at index 2, Charlie_Brown returned
5	Attendance statistics	Menu option 7	Average % and eligible/deficient counts	Pass — Avg 77.50%, 3 eligible (60%), 2 deficient (40%)
6	Merge Sort by roll no.	Menu option 9, algorithm 2	Vector remains ordered ascending by roll no.	Pass — order unchanged (already ascending), confirmed stable
7	Bubble Sort by name	Menu option 10, algorithm 1	Vector ordered alphabetically by name	Pass — Alice, Bob, Charlie, Diana, Ethan

#	Test Case	Input	Expected Result	Observed Result
8	Merge Sort by attendance %	Menu option 11, algorithm 2	Vector ordered ascending by attendance %	Pass — Diana(55) → Bob(70) → Ethan(80) → Alice(87.5) → Charlie(95)
9	Duplicate roll number rejected	Add Student, Roll No. 101	Operation refused with error message	Pass — “Student with Roll No 101 already exists.”
10	Save on exit	Menu option 14	students.txt overwritten with current data	Pass — “Saved 5 students to students.txt”

Table 3.2 — Functional test cases executed against the compiled application

3.3 Console Output — Registered Student Directory

The following is the unmodified console output produced by menu option 5 (Display Students) on program startup, confirming that all five seed records were parsed correctly, including the derived attendance percentage and eligibility columns.

```

-----
-----
Roll No   Name                Department   Total Classes   Attended   Attendance %
Eligibility
-----
-----
101      Alice_Smith          CSE          40             35         87.50
Eligible
102      Bob_Jones            ECE          40             28         70.00      Not
Eligible
103      Charlie_Brown        ME           40             38         95.00
Eligible
104      Diana_Prince         CSE          40             22         55.00      Not
Eligible
105      Ethan_Hunt           IT           40             32         80.00
Eligible
=====
=====
Total Students: 5

```

Fig. 3.1 — Captured output of the Display Students operation

3.4 Console Output — Low Attendance Report

Menu option 8 correctly isolates the two students below the 75% cutoff and computes how many additional classes each must attend (assuming no further classes are missed) to reach exactly 75%.

```

LOW ATTENDANCE REPORT (< 75% Cutoff)
-----
Roll No   Name                Attendance %   Deficit
-----
102      Bob_Jones           70.00         2

```

```
104      Diana_Prince      55.00      8
```

Fig. 3.2 — Captured output of the Low Attendance Report

3.5 Console Output — Binary Search

Searching for Roll No. 103 using Binary Search (menu option 4, algorithm 2) against the roll-number-sorted vector correctly resolves to index 2 and returns the Charlie_Brown record.

```
[Success] Student found at index 2!
```

```
-----
Roll No   Name           Department   Total Classes   Attended   Attendance %
Eligibility
-----
103      Charlie_Brown    ME           40              38         95.00
Eligible
```

Fig. 3.3 — Captured output of a successful Binary Search

3.6 Console Output — Attendance Statistics

```
SYSTEM ATTENDANCE STATISTICS
-----
Total Enrolled Students : 5
Average Attendance Rate : 77.50%
Eligible Students (>=75%): 3 (60.00%)
Deficient Students (<75%): 2 (40.00%)
```

Fig. 3.4 — Captured output of the Attendance Report & Stats operation

3.7 Console Output — Sorting Demonstrations

The same five-record dataset was sorted three times, once per available key, to verify both algorithms and all three SortCriteria values produce identical, correctly-ordered results.

3.7.1 Sort by Attendance Percentage (Merge Sort)

```
Roll No   Name           Department   Total Classes   Attended   Attendance %
Eligibility
-----
104      Diana_Prince    CSE          40              22         55.00      Not
Eligible
102      Bob_Jones       ECE          40              28         70.00      Not
Eligible
105      Ethan_Hunt      IT           40              32         80.00
Eligible
101      Alice_Smith     CSE          40              35         87.50
Eligible
103      Charlie_Brown   ME           40              38         95.00
Eligible
```

Fig. 3.5 — Result of Merge Sort keyed on attendance percentage (ascending)

3.7.2 Sort by Name (Bubble Sort)

Sorting the same directory alphabetically by name using Bubble Sort produced the sequence Alice_Smith, Bob_Jones, Charlie_Brown, Diana_Prince, Ethan_Hunt — which, incidentally, coincides with roll-number order for this particular seed dataset, confirming the algorithm's correctness on an input that happened to already be in the target order (triggering the early-exit optimisation after a single pass).

3.8 Comparative Discussion of Results

On the five-record seed dataset, the practical difference between Linear and Binary Search, and between Bubble Sort and Merge Sort, is not observable in wall-clock terms — all operations complete effectively instantly. The value of implementing both variants of each algorithm lies in demonstrating their differing asymptotic behaviour, which becomes significant as the student count n grows: for an institution-scale dataset of several thousand students, Binary Search's $O(\log n)$ cost remains a handful of comparisons where Linear Search's $O(n)$ cost grows linearly, and Merge Sort's guaranteed $O(n \log n)$ bound avoids Bubble Sort's $O(n^2)$ worst case entirely. Table 3.3 projects the number of elementary comparisons each algorithm would require as the dataset scales, to make this concrete.

n (students)	Linear Search (avg.)	Binary Search (max.)	Bubble Sort (worst)	Merge Sort (v)
5	~2.5	3	10	~12
100	~50	7	4,950	~664
1,000	~500	10	499,500	~9,966
10,000	~5,000	14	$\sim 4.99 \times 10^7$	~132,877

Table 3.3 — Projected comparison counts illustrating asymptotic growth

These projections make clear why the pairing of a hash-map index with binary search and merge sort is the correct engineering choice as the system scales beyond the small demonstration dataset used for this report, even though every algorithm gives visually identical results on five records.

3.9 Testing Methodology

Testing combined black-box functional testing (Section 3.2), driven purely by menu inputs and observed console output without reference to internal implementation, with a smaller set of targeted white-box checks aimed at the boundary conditions of the underlying algorithms and data structures — for example, verifying that Binary Search behaves safely when invoked on data that is not sorted, and that the low-attendance deficit calculation does not divide by zero for a newly added student with `totalClasses = 0`. Table 3.4 lists the edge-case and negative test cases exercised in addition to the core functional cases.

#	Edge Case	Expected Behaviour	Observed Result
E1	Search for a non-existent roll number	Both Linear and Binary Search return -1; “not found” message shown	Pass
E2	Binary Search attempted on	Warning printed; search proceeds	Pass — warning

#	Edge Case	Expected Behaviour	Observed Result
	unsorted data	but may be unreliable, by design	correctly triggered when order is disturbed
E3	Delete a roll number that does not exist	Operation refused with a clear error, no state change	Pass — “not found” error, no vector/file mutation
E4	Add a student with <code>attendedClasses > totalClasses</code>	Input rejected before reaching <code>AttendanceSystem</code>	Pass — caught in <code>main.cpp</code> validation layer
E5	Undo when the undo stack is empty	Informational message, no crash, no file write	Pass — “No attendance records to undo”
E6	Attendance percentage for <code>totalClasses = 0</code>	Returns 0.0 rather than dividing by zero	Pass — guarded in <code>getAttendancePercentage()</code>
E7	Sort an empty or single-record list	No crash; list returned unchanged	Pass — guarded by <code>mergeSort</code> 's <code>left >= right</code> base case
E8	Re-load after program restart	All previously saved records reappear unchanged	Pass — file round-trips losslessly

Table 3.4 — Edge-case and negative test cases

All observed results matched expected behaviour, giving confidence that the chosen data structures and algorithms are not only asymptotically appropriate (Section 2) but also correctly implemented and robust to the boundary conditions that most commonly cause defects in list-processing code: empty collections, missing keys, and division by a zero denominator.

3.10 Web Interface — Observed Behaviour

The companion browser dashboard was reviewed for functional parity with the console core. Its Sorting section successfully animates the Bubble Sort and Merge Sort algorithms step by step (highlighting each comparison, swap, and merge operation), its Search section reproduces both Linear and Binary Search with equivalent results to the C++ core, and its Reports section reproduces the same low-attendance deficit calculation, additionally offering a one-click CSV export of the report. Because the web layer persists data to the browser's `localStorage` rather than `students.txt`, the two implementations maintain separate datasets by design; this is discussed further as an integration opportunity in Section 6.

4. Conclusion and Future Scope

4.1 Conclusion

This project set out to build a working Student Attendance Management System while using its feature requirements as a natural framework for applying core Data Structures and Algorithms: a dynamic array and hash map for efficient storage and lookup, a queue for ordered batch processing, a stack for reversible undo, two searching algorithms of differing complexity, and two sorting algorithms of differing complexity across three distinct sort keys. All of these were implemented, compiled without errors, and exercised against the supplied dataset, with the resulting console output verified and reproduced in Section 3 of this report.

The system successfully satisfies its stated objectives: it registers, updates, and removes students; it processes batch attendance sessions in FIFO order with a working stack-based undo; it persists all data to a flat file that survives across program restarts; and it produces the two key administrative reports — a full directory and a low-attendance list with computed deficits — that motivated the project in the first place. The companion web dashboard extends the same conceptual model into an interactive, visual medium that is particularly effective for demonstrating how the sorting and searching algorithms behave step by step, which is difficult to convey from console text output alone.

Beyond the specific deliverable, the project reinforces a number of broader lessons: that the right choice of data structure for a given access pattern (hash map for lookup, queue for order-preserving processing, stack for reversibility) can be more consequential to a system's usability than any individual line of business logic; that asymptotic complexity, while invisible on small demonstration datasets, becomes decisive at realistic institutional scale; and that separating concerns into focused modules (entity, persistence, search, sort, controller, UI) keeps each algorithm testable and replaceable in isolation, as was done here.

4.2 Limitations

- Attendance is tracked as a single aggregate total per student rather than per subject or per date, so the system cannot currently answer questions like “what was this student's attendance in Operating Systems on 12 March?”
- Persistence relies on a flat text file with a fixed, whitespace-delimited schema; it has no transactional guarantees, no concurrent-access protection, and student names cannot contain spaces (the sample data uses underscores as a workaround).
- The undo mechanism supports only the single most recent attendance action per session (the stack is not persisted to disk), rather than a full multi-level history across restarts.
- The system has no authentication or access-control layer; any user with console or file access has full read/write privileges over all student records.
- The C++ console core and the browser-based web interface are functionally parallel but operate on entirely separate datasets (file-based vs. localStorage-based), so they cannot currently be used interchangeably against the same live data.

4.3 Future Scope

- Migrate persistence from a flat file to a relational database (e.g., SQLite or PostgreSQL) to gain transactional integrity, concurrent access, and richer querying, while retaining the same in-memory data-structure design for the hot path.
- Extend the Student model to per-subject, per-session attendance records, enabling subject-wise and date-wise reporting in addition to the current aggregate view.
- Expose the C++ AttendanceSystem through a REST API (or recompile it to WebAssembly) so the existing web dashboard can operate directly against the C++ core's logic and data instead of maintaining an independent JavaScript re-implementation.
- Extend the undo stack into a persisted, multi-level action history/audit log, and add a corresponding “redo” capability.
- Introduce role-based authentication (administrator vs. faculty vs. read-only viewer) before any production deployment.
- Add automated unit tests (e.g., with Catch2 or GoogleTest) for the Search, Sort, and AttendanceSystem modules to formalise the manual testing carried out in Section 3.

Appendix A: Key Source Code Excerpts

This appendix reproduces short excerpts of the project's own source code (as supplied for this report) to illustrate, alongside the pseudocode of Section 2, exactly how the core data structures and algorithms were realised in C++.

A.1 Student Entity — Safe Percentage Calculation

```
double Student::getAttendancePercentage() const {
    if (totalClasses <= 0) {
        return 0.0;                // guards against division by zero
    }
    return (static_cast<double>(attendedClasses) / totalClasses) * 100.0;
}
```

A.2 Rebuilding the Roll-Number Hash Index

```
void AttendanceSystem::rebuildRollIndex() {
    rollIndex.clear();
    for (size_t i = 0; i < students.size(); ++i) {
        rollIndex[students[i].getRollNo()] = static_cast<int>(i);
    }
}
```

A.3 Binary Search (Search.h)

```
inline int binarySearch(const std::vector<Student>& sortedStudents, int rollNo) {
    int low = 0;
    int high = static_cast<int>(sortedStudents.size()) - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        int midRoll = sortedStudents[mid].getRollNo();
        if (midRoll == rollNo) return mid;
        else if (midRoll < rollNo) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}
```

A.4 Merge Step of Merge Sort (MergeSort.h)

```
inline void merge(std::vector<Student>& students, int left, int mid, int right, SortCriteria
criteria) {
    int n1 = mid - left + 1;
    int n2 = right - mid;
    std::vector<Student> L(n1), R(n2);
    for (int i = 0; i < n1; ++i) L[i] = students[left + i];
    for (int j = 0; j < n2; ++j) R[j] = students[mid + 1 + j];

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
```



```

        if (compareStudents(L[i], R[j], criteria)) students[k++] = L[i++];
        else students[k++] = R[j++];
    }
    while (i < n1) students[k++] = L[i++];
    while (j < n2) students[k++] = R[j++];
}

```

A.5 Queue-Driven Batch Attendance Marking (excerpt)

```

std::queue<int> q;
for (int roll : rollNumbers) q.push(roll);

while (!q.empty()) {
    int roll = q.front(); q.pop();
    int index = findIndexByRollNo(roll);
    if (index == -1) { /* warn and skip */ continue; }
    // ...prompt for Present/Absent, update counters...
    undoStack.push({roll, status}); // record for undo
}
if (modified) saveToFile(); // single write for the whole batch

```

A.6 Stack-Driven Undo (excerpt)

```

auto lastAction = undoStack.top();
undoStack.pop();
int roll = lastAction.first;
std::string status = lastAction.second;
int index = findIndexByRollNo(roll);
if (index != -1) {
    Student& s = students[index];
    if (s.getTotalClasses() > 0) s.setTotalClasses(s.getTotalClasses() - 1);
    if (status == "Present" && s.getAttendedClasses() > 0)
        s.setAttendedClasses(s.getAttendedClasses() - 1);
}
saveToFile();

```

Complete, unabridged source files (Student.h/.cpp, AttendanceSystem.h/.cpp, FileManager.h, Search.h, MergeSort.h, main.cpp) were compiled and executed as described in Section 3.1, and are available alongside this report as the original project deliverable.

5. References

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. Introduction to Algorithms (3rd ed.). MIT Press.
2. Sedgewick, R., & Wayne, K. Algorithms (4th ed.). Addison-Wesley.
3. Stroustrup, B. The C++ Programming Language (4th ed.). Addison-Wesley.
4. ISO/IEC 14882:2017 — Programming Languages — C++ (C++17 Standard).
5. cppreference.com — C++ Standard Library reference: `std::vector`, `std::unordered_map`, `std::queue`, `std::stack`. Retrieved 2026.
6. Mozilla Developer Network (MDN) Web Docs — HTML, CSS, and JavaScript reference documentation. Retrieved 2026.
7. GeeksforGeeks — Articles on Bubble Sort, Merge Sort, Linear Search, and Binary Search complexity analysis. Retrieved 2026.
8. Project source files: `Student.h/.cpp`, `AttendanceSystem.h/.cpp`, `FileManager.h`, `Search.h`, `MergeSort.h`, `main.cpp`, `students.txt`, `index.html`, `style.css`, `app.js` (as supplied for this report).

github—<https://github.com/Gauravjha108/Student-Attendance-Management-System>